Functions over Structured Data

Lecture 27

CS 1001

Kirk Duran

Today

- Design functions whose inputs and outputs are lists, nested lists, strings, and dictionaries.
- Decompose structured-data problems into smaller transformations and summaries.
- Use selected Python tools—`sum`, `max`, `enumerate`, `zip`, `map`, `filter`, `any`, and `all`—when they express the intended computation directly.
- Apply `while` to repetition whose number of iterations is not known in advance.
- Preserve the distinction between producing a new structure and mutating an existing one.

Key idea

Today is a design lecture: each problem begins as a prompt, and we choose a representation and control structure that match the data and the required result.

The Boundary for This Lecture

Today

- functions over structured values
- decomposition and helper functions
- selected sequence built-ins
- `map` and `filter`
- `enumerate` and `zip`
- `while`
- Boolean aggregation with `any` / `all`

Not today

- searching algorithms
- sorting algorithms
- algorithmic complexity
- object-oriented design
- classes or methods we define ourselves
- advanced library APIs

Part 1: Prompt, Design, Implement, Refine

How Today's Lecture Works

For each problem we will follow the same sequence.

Prompt $\longrightarrow$ **Data Contract** $\longrightarrow$ **Plan** $\longrightarrow$ **Code** $\longrightarrow$ **Refine**

- First decide what the input and output structures mean.
- Then identify the repeated computation or changing state.
- Write a correct version using mechanisms we already understand.
- Finally ask whether a Python construct communicates that same intent more directly.

Key idea

A built-in is useful when it removes incidental mechanism without hiding the computation we need to understand.

Structured Data Appears in the Function Contract

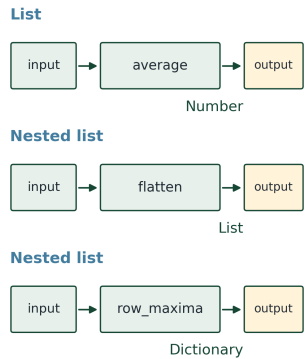
Examples

```
def average(values:  list[float]) -> float:

def flatten(grid:  list[list[int]]) -> list[int]:

def row_maxima(grid:  list[list[int]]) -> dict[int,
int]:
```

- The annotation communicates more than element type.
- It communicates the **shape of the expected data**; annotations do not enforce types or row lengths at runtime.
- The return type states what representation the function promises to produce.



Prompt 1: Compute an Average

In class

Write a function that accepts a nonempty list of numerical values and returns their arithmetic mean.

Contract

```
def average(values:  list[float]) -> float:
```

Before writing code:

1. What quantity must be accumulated?
2. What second quantity is needed to produce the mean?
3. Does the input list need to be modified?

A Correct Loop-Based Solution

Mechanism made explicit

```
def average(values:  list[float]) -> float:
   total = 0.0
   for value in values:
      total = total + value
   return total / len(values)
```

- `total` is the result-so-far accumulator.
- `len(values)` supplies the number of observations.
- The input is read but never mutated.

When the Built-In Matches the Intent

Python already provides `sum` for numerical accumulation.

Refined implementation

```
def average(values:  list[float]) -> float:
   return sum(values) / len(values)
```

explicit accumulation $\longrightarrow$ named aggregate operation

Key idea

The function contract and mathematical meaning have not changed. Only the level at which the accumulation is expressed has changed.

Prompt 2: Average of Averages

In class

A nested list stores several groups of scores. Return the average of the average score from each group.

Example input

```
groups = [[80, 90], [70, 100], [85, 95]]
```

Desired intermediate structure:

```
[85.0, 85.0, 90.0]
```

Then average those three values. Assume the outer list and every group are nonempty.

Decompose the Problem with a Helper

Reuse the abstraction we just designed

```
def average(values:  list[float]) -> float:
   return sum(values) / len(values)

def average_of_averages(groups:  list[list[float]]) -> float:
   group_averages = []
   for group in groups:
      group_averages.append(average(group))
   return average(group_averages)
```

Key idea

The outer function reasons about groups. The helper function owns the lower-level meaning of "average one group."

Average of Averages Is a Specific Contract

Suppose groups have different lengths:

Data

```
[[100], [0, 0, 0]]
```

Average of group averages

```
(100 + 0) / 2 = 50
```

Each group receives equal weight.

Average of all observations

```
(100 + 0 + 0 + 0) / 4 = 25
```

Each observation receives equal weight.

Design consequence

A function name and purpose statement must say which quantity is intended.

Part 2: Functions over Nested Structures

Prompt 3: Flatten One Level

In class

Write a function that accepts a two-dimensional list and returns a new one-dimensional list containing the elements row by row.

Example

```
[[1, 2], [3, 4, 5], [6]]
⟶
[1, 2, 3, 4, 5, 6]
```

Questions:

1. How many structural levels must be traversed?
2. What state should accumulate the result?
3. Should any row in the input be modified?

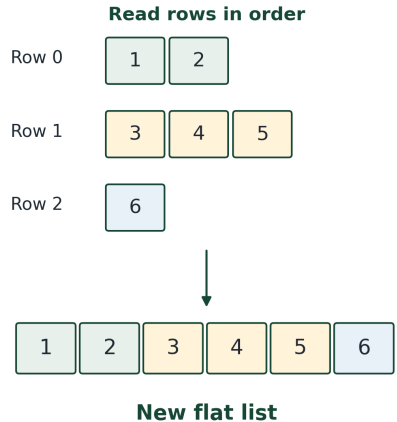
Flattening Requires Traversing Two Structural Levels

outer list → row → element

- The outer loop selects one row at a time.
- The inner loop selects one element from that row at a time.
- A fresh output list accumulates the elements in encounter order.

Key idea

Nested data usually leads to nested processing because each loop traverses one structural level.



Implementation: Flatten One Level

Function

```
def flatten(grid:  list[list[int]]) -> list[int]:
   result = []
   for row in grid:
      for value in row:
         result.append(value)
   return result
```

- `grid` and its rows are read only.
- `result` is intentionally mutated because it is a new list owned by this function.
- The returned structure is independent of the outer input list.

Prompt 4: Record the Maximum of Each Row

In class

Given a two-dimensional list of integers, return a dictionary mapping each row index to the maximum value in that row.

Example

```
[[4, 9, 2], [7, 1, 8], [5, 3, 6]]
⟶
{0:  9, 1:  8, 2:  6}
```

Assume every row is nonempty, as required by `max`. This problem needs both:

- each row's **position**, and
- the row **value** itself.

`enumerate`: Position and Value Together

Compare:

Index-driven

```
for i in range(len(rows)):
   row = rows[i]
```

Enumerated

```
for i, row in enumerate(rows):
   ...
```

Definition

`enumerate(sequence)` supplies pairs containing an element's index and the element value.

Use it when the computation genuinely needs both pieces of information.

Implementation: Row Maxima

Function

```
def row_maxima(grid:  list[list[int]]) -> dict[int, int]:
   result = {}
   for index, row in enumerate(grid):
      result[index] = max(row)
   return result
```

row position + row aggregate $\longrightarrow$ dictionary association

Key idea

The output representation is part of the solution: the dictionary preserves an explicit association between each row index and its summary value.

Part 3: Named Transformation Patterns — `map` and `filter`

Prompt 5: Apply One Function to Every Value

In class

Given a list of integers, produce a new list containing the square of each value.

We already know two representations:

Explicit loop

```
result = []
for value in values:
   result.append(square(value))
```

Comprehension

```
result = [square(value) for value in values]
```

`map`: Name the Transformation Pattern Directly

Helper function

```
def square(value:  int) -> int:
   return value * value
```

Mapping

```
result = list(map(square, values))
```

Definition

`map(function, sequence)` applies the function to each supplied value. In Python 3, `map` produces an iterator: values are generated as it is consumed. `list(...)` collects them into a list.

same computational pattern — different representation

`filter`: Name the Selection Pattern Directly

Predicate

```
def positive(value:  int) -> bool:
   return value > 0
```

Filtering

```
result = list(filter(positive, values))
```

Definition

`filter(predicate, sequence)` retains the values for which the predicate returns `True`.

The predicate decides **whether** an existing value remains; it does not transform that value. Like `map`, `filter` returns an iterator.

Loop, Comprehension, `map`, or `filter`?

Representation	Good fit
explicit `for`	multiple statements or changing state
comprehension	compact transform / select into a new list
`map`	apply an existing named function to every value
`filter`	retain values using an existing named predicate

Key idea

These forms are not ranked from “basic” to “advanced.” Choose the representation that makes the data relationship easiest to read.

Prompt 6: Caesar Cipher

In class

Write a function that shifts each lowercase English letter forward by a fixed number of positions, wrapping from `z` back to `a`.

Example with shift 3

`"attack"` $\longrightarrow$ `"dwwdfn"`

Assume the input contains only lowercase letters `a` through `z`. This combines ideas from several units:

- strings as immutable sequences,
- `ord` / `chr`,
- modular arithmetic,
- iteration and function decomposition.

Design a One-Character Transformation First

For lowercase letters, `ord('a')` provides a useful origin.

Helper

```
def shift_char(char:  str, shift:  int) -> str:
   offset = ord(char) - ord("a")
   shifted = (offset + shift) % 26
   return chr(ord("a") + shifted)
```

character $\rightarrow$ numeric offset $\rightarrow$ shifted offset $\rightarrow$ character

Build the String from Transformed Characters

Function

```
def caesar(text:  str, shift:  int) -> str:
   result = ""
   for char in text:
      result = result + shift_char(char, shift)
   return result
```

Key idea

Because strings are immutable, each update rebinds `result` to a newly produced string. The input `text` is never mutated.

Part 4: Functions over Corresponding Structures

Prompt 7: Match Student Names with Grades

In class

Two lists store corresponding data: one contains names and the other contains scores. Return the names of students whose score is at least 70.

Data

```
names = ["Ada", "Grace", "Alan", "Edsger"]
scores = [92, 68, 81, 65]
```

Desired result:

```
["Ada", "Alan"]
```

`zip`: Corresponding Values Together

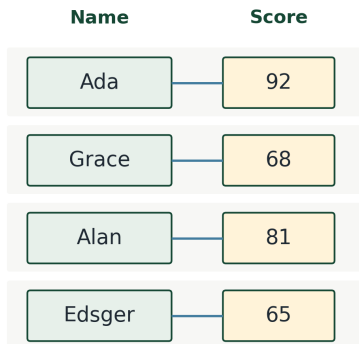
Iteration

```
for name, score in zip(names, scores):
    ...
```

Definition

`zip` groups values from corresponding positions of multiple sequences.

- first name with first score,
- second name with second score,
- and so on.



One corresponding pair per iteration

Implementation: Passing Students

Function

```
def passing_students(
   names:  list[str], scores:  list[int]
) -> list[str]:
   result = []
   for name, score in zip(names, scores):
      if score >= 70:
         result.append(name)
   return result
```

Function contract

This design assumes equal lengths and corresponding data in the same order. By default, `zip` stops at the shorter input; check lengths when every entry must have a partner.

Part 5: When Repetition Depends on a Condition

Prompt 8: Keep Asking Until the Answer Is Correct

In class

Ask the user `"What is 4 + 5?"`. If the answer is wrong, ask again. Stop only after the user enters 9.

Assume each response can be converted to an integer. The number of repetitions is unknown:

- the user may answer correctly immediately;
- the user may require several attempts;
- there is no input sequence whose length determines the repetition count.

Key idea

This is condition-controlled repetition rather than sequence-controlled iteration.

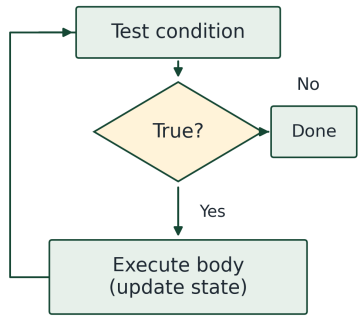
The `while` Statement

General form

```
while condition:
    body
```

Execution model:

1. evaluate the condition;
2. if `False`, leave the loop;
3. if `True`, execute the body;
4. return to the condition and evaluate it again.



Test again after every body

Implementation: Repeated Math Prompt

Function

```
def ask_addition() -> None:
   answer = int(input("What is 4 + 5?  "))
   while answer != 9:
      print("Try again.")
      answer = int(input("What is 4 + 5?  "))
   print("Correct.")
```

Key idea

Something in the loop body must make progress toward changing the condition. Here, a new user response replaces the previous value of `answer`.

Choose `for` or `while` from the Source of Repetition

Question	Typical construct	Example
process each value in a sequence	`for`	each row in a grid
repeat a known count	`for` + `range`	five trials
repeat while a condition holds	`while`	ask until correct

Common failure mode

A `while` loop can run forever if its condition never becomes `False`.

Part 6: Climax — Password Validation with `any` and `all`

Final Prompt: Validate a Password

In class

Write a function that returns `True` exactly when a password:

- has at least 8 characters,
- contains at least one uppercase letter,
- contains at least one lowercase letter,
- contains at least one digit, and
- contains no whitespace characters.

Contract

```
def valid_password(password:  str) -> bool:
```

`any`: Does At Least One Result Satisfy the Condition?

Suppose we compute one Boolean per character:

Uppercase checks

```
[char.isupper() for char in password]
```

Definition

`any(values)` returns `True` when at least one supplied value is truthy.

Therefore:

Requirement

```
has_upper = any([char.isupper() for char in password])
```

The same pattern works for lowercase letters and digits. `any([])` is `False`; here the comprehension builds its Boolean list before `any` combines the results.

`all`: Does Every Result Satisfy the Condition?

For whitespace, the requirement applies to **every** character.

Per-character claim

```
not char.isspace()
```

Definition

`all(values)` returns `True` when every supplied value is truthy.

Therefore:

Requirement

```
no_space = all([not char.isspace() for char in password])
```

`any`: existential condition `all`: universal condition

`all([])` is `True`: no element violates the condition.

Build Each Requirement Independently

Intermediate Boolean values

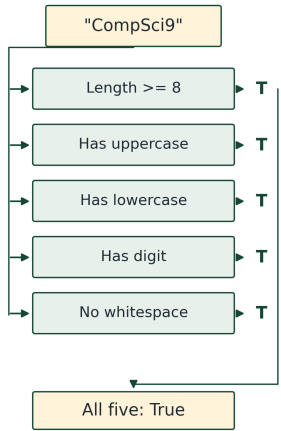
```
long_enough = len(password) >= 8
has_upper = any([char.isupper() for char in password])
has_lower = any([char.islower() for char in password])
has_digit = any([char.isdigit() for char in password])
no_space = all([not char.isspace() for char in password])
```

Key idea

Instead of one complicated expression, name each independent requirement so its meaning can be inspected and tested separately.

Complete Password Checker

```
def valid_password(password: str) -> bool:
    long_enough = len(password) >= 8
    has_upper = any([char.isupper()
                     for char in password])
    has_lower = any([char.islower()
                     for char in password])
    has_digit = any([char.isdigit()
                     for char in password])
    no_space = all([not char.isspace()
                     for char in password])
    return (long_enough and has_upper and
            has_lower and has_digit and no_space)
```



Trace the Requirements, Not Just the Final Boolean

Password	Len	Upper	Lower	Digit	No space	Valid
`CompSci9`	T	T	T	T	T	T
`compsci9`	T	F	T	T	T	F
`Comp Sci9`	T	T	T	T	F	F
`CS9abc`	F	T	T	T	T	F

Key idea

A complex validation function becomes easier to reason about when each rule is represented by one named Boolean claim.

The Structured-Data Toolkit We Used Today

Tool	Question it answers
`sum` / `max`	how can this structure be summarized?
`enumerate`	what are the position and value together?
`zip`	which values from different structures correspond?
`map`	what should every value become?
`filter`	which existing values should remain?
`any`	does at least one Boolean condition hold?
`all`	do all Boolean conditions hold?
`while`	should repetition continue based on current state?

Takeaways

1. The shape and meaning of structured data belong in a function's contract.
2. Decompose a large structured-data problem into smaller transformations and summaries.
3. Use nested iteration when the data itself has nested structure.
4. Use `enumerate` when a computation needs position and value; use `zip` when it needs corresponding values from multiple sequences.
5. `map` and `filter` are named forms of transformation and selection patterns already studied with loops and comprehensions.
6. Use `while` when repetition is governed by a condition rather than by a sequence.
7. `any` and `all` let many Boolean results collapse into one higher-level decision.

Next

The next unit introduces **objects and compound data**: a new way to group related state under a single programmer-defined abstraction.